

Checkpoint V: Third Prototype

Group: G16

Date: 2025/10/12

Prototype Architecture

The project continues to focus on **comparative analysis of Pokémon card prices** across multiple sets, years, series, and types. The goal remains to support collectors and researchers in exploring how market dynamics and card attributes affect value and popularity.

In this checkpoint, the main architectural evolution was the **implementation and integration of the Violin Plot idiom**, which completes the initially proposed set of four charts (Scatterplot, Bar Chart, Line Chart, Violin Plot).

The overall system structure remains modular and event-driven, but several refinements were introduced to extend the scalability and coherence of the architecture:

1. ViolinPlot.js Module

- Introduced as a standalone idiom component following the same structure as existing chart modules (e.g., createScatterplot, createLineChart).
- Uses Enter–Update–Exit patterns and modularized rendering (axes, violins, circles) to ensure consistency with the rest of the architecture.
- The module is fully integrated into the shared state mechanism (selectedCards, updateSelectionAcrossPlots()), ensuring that interaction logic remains unified.

2. Density Computation Layer

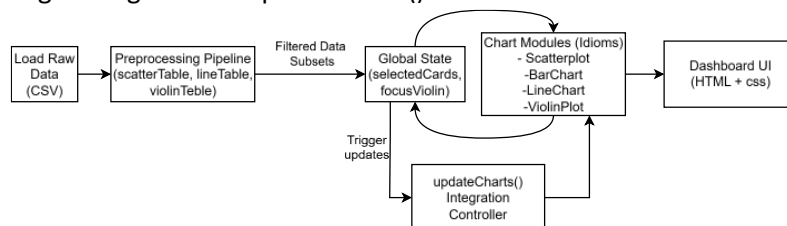
- A new **kernel density estimation (KDE)** function was added inside ViolinPlot.js to compute the smoothed probability distribution of card prices by rarity.
- This introduces a lightweight statistical layer built directly with D3 (d3.line, d3.area, d3.curveCatmullRom) and avoids reliance on any external libraries.

3. Unified Interaction API

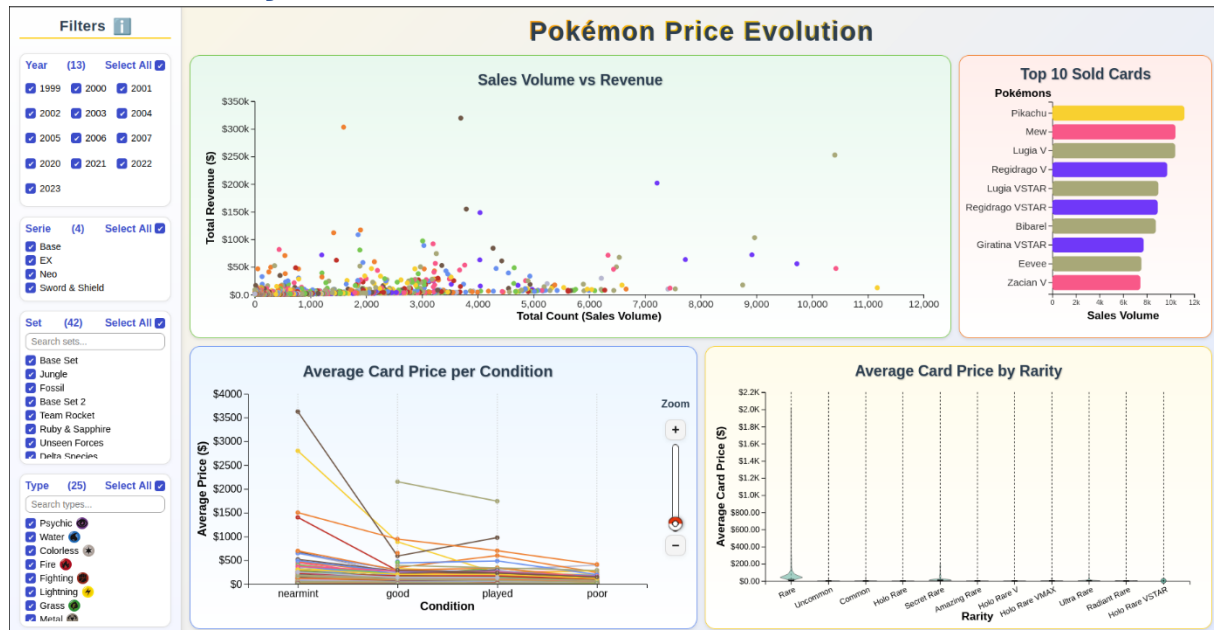
- The event handlers (mouseover, mouseout, click) now follow a common structure across all idioms.
- A focusCard state variable was added to distinguish between temporary hover highlights and persistent multi-selection, providing a more predictable user experience.

4. Improved Scalability

- The architecture remains fully compatible with future idioms due to the **shared global filtering pipeline** and **cross-module update functions**.
- Adding a new chart now only requires defining its local render/update functions and registering them in updateCharts().



Dashboard Layout



The overall dashboard layout remains **identical in structure** to Checkpoint IV: filters occupy the left panel and the visualization grid on the right contains a 2×2 matrix of idioms.

However, the **Violin Plot** now replaces the placeholder from CP IV, completing the lower row of the grid (bottom-right corner). This addition introduces visual balance and thematic diversity:

- Top row → *quantitative comparisons* (Scatterplot + Bar Chart).
- Bottom row → *temporal and statistical variation* (Line Chart + Violin Plot).

Minor layout refinements include:

1. **Responsive Resizing** – chart containers now dynamically adapt to window width, ensuring consistent margins and axis readability.
2. **Legend Consolidation** – a shared legend below the grid uses consistent color encoding for card rarities, unifying the visual vocabulary across all idioms.
3. **Help and Tooltip Positioning** – improved placement prevents tooltips from overlapping with dense violin shapes.

Data Processing

Checkpoint V introduced additional preprocessing steps to support the Violin Plot, which relies on distribution data rather than simple aggregations.

1. **Rarity-Based Aggregation**
 - A new `violinTable` is created during data load, grouping cards by rarity and collecting all price values.
 - This table is then used to compute density curves per rarity level using kernel density estimation (KDE).
2. **Kernel Density Estimation (KDE)**
 - Implemented using D3's built-in array utilities (`d3.range`, `d3.mean`) and custom Gaussian kernels.
 - This produces smooth violin shapes showing the spread and concentration of card prices.
 - The computation runs once during data initialization and is filtered at runtime, ensuring high performance.
3. **Filter Compatibility**
 - The `violinTable` is sliced based on the same filter logic as other idioms (set, type, year).

- This allows the Violin Plot to instantly reflect any subset of the dataset without recalculating the kernel.

Chart Interaction

The Violin Plot introduces new interactive and dynamic behaviours consistent with D3’s Enter–Update–Exit pattern. While the visual idiom differs from the others, the underlying principles—smooth perceptual transitions, contextual highlighting, and consistent selection semantics—remain unchanged.

Enter Phase

When filters change or the dashboard is first initialized, new violins are created for each rarity class present in the filtered data.

Update Phase

When the filtered dataset changes (for instance, due to year or type filters), violins already present are updated instead of being redrawn:

- **Rescaling Axes:** The x-scale domain (price) adjusts dynamically to the new maximum price range, and both the axis and violins transition in sync to maintain contextual stability.

Exit Phase

When a rarity category is no longer present (e.g., filters exclude it), its violin is removed from the DOM:

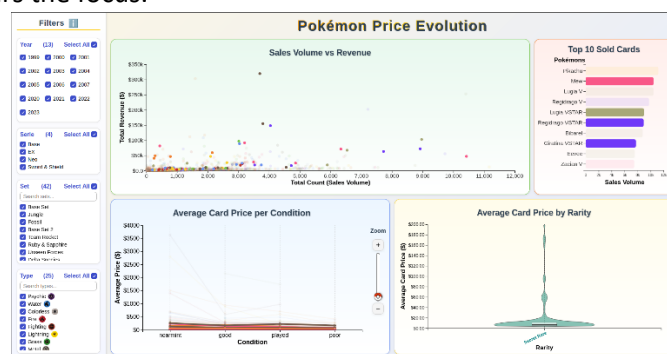
- The fill opacity transitions to zero, and the shape contracts toward the centre line before removal.
- Associated tooltip and event bindings are also cleared, maintaining memory efficiency.

This controlled exit behaviour is similar to the one used in the other idioms, ensuring all elements in the dashboard behave the same way.

Interactive Features

Beyond the lifecycle mechanics, the Violin Plot also introduces fine-grained interactions:

- **Hover:** Highlights the violin, increases opacity, and brings it to the front using z-order changes.
- **Tooltip:** Displays distribution statistics (rarity, median, min–max range, sample size).
- **Click-to-Focus:** Clicking a violin/box/axis text locks it in a “focused” state, placing it alone on the charts and with a scale that better fits that individual violin. Clicking again or clicking elsewhere clears the focus.



- **Linked Highlighting:** When a Violin is in “focus” all the cards that create that violin are highlighted in the other charts.

Chart Integration

The Violin Plot was fully integrated into the global coordination architecture established in Checkpoint IV.

1. Shared Global State

- The selectedCards and focusCard variables remain the single sources of truth for selection status.

- All idioms—including the new Violin Plot—subscribe to these shared states and trigger synchronized updates.
- 2. **Bidirectional Linking**
 - The linking mechanism now extends to distribution-level data: selecting a violin updates scatterplot points, bar bars, and line paths corresponding to the selected rarity or card type.
- 3. **Update Flow**
 - The `updateCharts()` controller was expanded to include the `updateViolinPlot()` call, maintaining the same unified refresh cycle.
 - This modular integration ensures that adding future idioms would only require registering a new update function, with no structural changes.
- 4. **Performance Consistency**
 - Thanks to precomputed `violinTable` and efficient filtering, the entire dashboard remains responsive, even with multiple linked highlights and rapid filter toggling.